

Лабораторная работа №7

Дискретно-событийное моделирование

Люпп Софья Романовна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
5	Выводы	19
	Список литературы	20

Список иллюстраций

4.1	Создание файла simulation.jl в src	9
4.2	Создание скрипта run_simulation.jl	10
4.3	Создание скрипта run_simulation.jl	11
4.4	Pkg.Add	12
4.5	Julia для скриптов	12
4.6	Julia для скриптов	12
4.7	Julia для скриптов	13
4.8	Вывод файла run_simulation.jl	13
4.9	График времени, проведенного клиентом (заявкой) в системе . .	14
4.10	График времени ожидания обслуживания	14
4.11	Гистограмма	15
4.12	Модель Росса скрипт в src	16
4.13	Вывод реализации в консоль	16
4.14	График для разного количества машин	17
4.15	Вывод реализации в консоль	17
4.16	График изменения числа исправных машин во времени	18

Список таблиц

1 Цель работы

Изучить дискретно-событийное моделирование и реализовать модель Росса

2 Задание

- 1) Изучить теорию по дискретно-событийному моделированию и модель Росса;
- 2) Создать необходимые скрипты по заданию;
- 3) Выполнить отчет и презентацию, преобразовать документацию в формате Quarto и выполнить литературный код.

3 Теоретическое введение

Модель М/М/с (по классификации Кендалла) — это система массового обслуживания со следующими свойствами: — М (Markovian) — входящий поток заявок пуассоновский, интервалы между прибытиями распределены экспоненциально с параметром λ . — М — время обслуживания каждой заявки распределено экспоненциально с параметром μ . — с — количество идентичных обслуживающих приборов (каналов), работающих параллельно.

Модель Росса же представляет собой классический пример системы массового обслуживания с конечной популяцией, резервом и ремонтом.

— В системе находятся N идентичных машин, которые постоянно работают и могут выходить из строя. — R машин находятся в резерве и готовы немедленно заменить любую отказавшую. — Одно ремонтное устройство (ремонтник), которое может одновременно ремонтировать только одну машину. — Когда работающая машина ломается, происходит следующее: — Немедленно берётся одна резервная машина (если она есть) и запускается в работу вместо сломавшейся. — Сломанная машина отправляется в ремонт. — Если резерва нет, система падает (crash). Моделирование заканчивается. — После ремонта машина пополняет пул резервных (становится исправной и ждёт). — Требуется оценить среднее время до падения системы $E[T]$ при заданных распределениях наработки до отказа и времени ремонта.

4 Выполнение лабораторной работы

Начинаю лабораторную с главного скрипта, где будет реализована вычислительная логика модели М/М/С (рис. 4.1).


```

1 module SimulationModel
2 using StableRNGs
3 using Distributions
4 using ConcurrentSim
5 using DataFrames
6 export run_simulation
7 const rng = StableRNG{123}
8 @resumable function customer(
9     env::Environment,
10    server::Resource,
11    id::Int,
12    arrival_time::Float64,
13    service_dist::Distribution,
14    results::Vector
15 )
16
17     @yield timeout(env, arrival_time)
18
19     arrival = now(env)
20
21     req = request(server)
22     @yield req
23
24     service_start = now(env)
25     wait_time = service_start - arrival
26
27     service_time = rand(rng, service_dist)
28
29     @yield timeout(env, service_time)
30
31     departure = now(env)
32
33     @yield unlock(server)
34
35     push!(results, (
36         id = id,
37         arrival = arrival,
38         service_start = service_start,
39         departure = departure,
40         wait_time = wait_time,
41         service_time = service_time,
42         system_time = departure - arrival
43     ))
44 end
45
46 function run_simulation(

```

Рисунок 4.1: Создание файла simulation.jl в src

После чего в scripts создаю файл run_simulation.jl, который реализует модель с двумя обслуживающими приборами и с заданными интенсивностями обслуживания и прибытия (рис. 4.2). Сохраняю графики в plots (рис. 4.3).

```

1 using DrWatson
2 @quickactivate "MMC"
3 using CairoMakie
4 using CSV
5 using DataFrames
6 include(sourcedir("simulation.jl"))
7 using .SimulationModel
8
9 # параметры модели
10 num_customers = 1000
11 num_servers = 2
12
13 λ = 0.9
14 μ = 0.5
15
16 df = run_simulation(
17     num_customers,
18     num_servers,
19     λ,
20     μ
21 )
22
23 # сохранение результатов
24 CSV.write(datadir("sims/results.csv"), df)
25
26 println(first(df, 5))
27
28 fig1 = Figure()
29
30 ax1 = Axis(
31     fig1[1,1],
32     title = "Waiting Time",
33     xlabel = "Customer ID",
34     ylabel = "Wait Time"
35 )
36
37 lines!(ax1, df.id, df.wait_time)
38
39 save(plotsdir("waiting_time.png"), fig1)
40
41 fig2 = Figure()
42
43 ax2 = Axis(
44     fig2[1,1],
45     title = "System Time",
46     xlabel = "Customer ID",

```

Рисунок 4.2: Создание скрипта run_simulation.jl

```

11 )
12
13 # сохранение результатов
14 CSV.write(datadir("sims/results.csv"), df)
15
16 println(first(df, 5))
17
18 fig1 = Figure()
19
20 ax1 = Axis(
21     fig1[1,1],
22     title = "Waiting Time",
23     xlabel = "Customer ID",
24     ylabel = "Wait Time"
25 )
26
27 lines!(ax1, df.id, df.wait_time)
28
29 save(plotsdir("waiting_time.png"), fig1)
30
31 fig2 = Figure()
32
33 ax2 = Axis(
34     fig2[1,1],
35     title = "System Time",
36     xlabel = "Customer ID",
37     ylabel = "Time in System"
38 )
39
40 lines!(ax2, df.id, df.system_time)
41
42 save(plotsdir("system_time.png"), fig2)
43
44
45 fig3 = Figure()
46
47 ax3 = Axis(
48     fig3[1,1],
49     title = "Histogram of Waiting Time",
50     xlabel = "Wait Time",
51     ylabel = "Frequency"
52 )
53
54 hist!(ax3, df.wait_time)
55
56 save(plotsdir("wait_histogram.png"), fig3)

```

Рисунок 4.3: Создание скрипта run_simulation.jl

Для этого в Julia добавляю необходимые пакеты (рис. 4.4) (рис. 4.5) (рис. 4.6) (рис. 4.7).

```

srluipp@vbox:~$ julia

      _       _       _
     ( )     ( )     ( )
    _ _ _   _ _ _   _ _ _
   | | | | | | | | | | | | |
   | | | | | | | | | | | |
   |_|_|_|_|_|_|_|_|_|_|_|_|
  _/ | \ _ _ ' _ | _ | \ _ _ ' _ |
 | _/ | _/

Documentation: https://docs.julialang.org
Type "?" for help, "]"? for Pkg help.
Version 1.10.2 (2024-03-01)
Official https://julialang.org/ release

julia> import Pkg; Pkg.add("StableRNGs")
Updating registry at `~/.julia/registries/General.toml`
Downloading [=====>] 22.1 %%
```

Рисунок 4.4: Pkg.Add

```

julia> Pkg.add("Pkg.instantiate()")
ERROR: `Pkg.instantiate()` is not a valid package name

Installed Static _____ v1.3.1
Installed EpollShim_jll _____ v0.0.20230411+1
Installed FilePaths _____ v0.9.0
Installed Xorg_libXau_jll _____ v1.0.13+0
Installed Distances _____ v0.10.12
Installed Missings _____ v1.2.0
Installed RoundingEmulator _____ v0.2.1
Installed OrdinaryDiffEqVerner _____ v1.11.0
Installed FiniteDiff _____ v2.30.0
Installed JLD2 _____ v0.6.4
Installed RealDot _____ v0.1.0
Installed MuladdMacro _____ v0.2.4
Installed SIMDTypes _____ v0.1.0
Installed FFMPEG _____ v0.4.5
Installed TriplotBase _____ v0.1.0
Installed SIMD _____ v3.7.2
Installed Showoff _____ v1.0.3
Installed IrrationalConstants _____ v0.2.6
Installed FastPower _____ v1.3.1
Installed Giflib_jll _____ v5.2.3+0
Installed OnlineStatsBase _____ v1.7.1
```

Рисунок 4.5: Julia для скриптов

```

julia> Pkg.add([
    "ConcurrentSim",
    "StableRNGs",
    "Distributions",
    "DataFrames",
    "CSV",
    "CairoMakie"
])
Resolving package versions...
```

Рисунок 4.6: Julia для скриптов

```
julia> import Pkg; Pkg.add("CairoMakie")
Resolving package versions...
Installed Sixel _____ v0.1.5
Installed AdaptivePredicates _____ v1.2.0
Installed PNGFiles _____ v0.4.4
Installed ImageIO _____ v0.6.9
Installed Unitful _____ v1.28.0
Installed JpegTurbo _____ v0.1.6
Installed EarCut_jll _____ v2.2.4+0
Installed ProgressMeter _____ v1.11.0
Installed TiffImages _____ v0.11.9
Installed OpenEXR _____ v0.3.3
Installed BaseDirs _____ v1.3.2
Installed Netpbm _____ v1.1.1
Installed SignedDistanceFields _____ v0.4.1
Installed AxisArrays _____ v0.4.8
Installed TriplotBase _____ v0.1.0
Installed RoundingEmulator _____ v0.2.1
Installed SIMD _____ v3.7.2
Installed FilePaths _____ v0.9.0
Installed Giflib_jll _____ v5.2.3+0
Progress [=====> ] 19/70
```

Рисунок 4.7: Julia для скриптов

Вывод файла run_simulation.jl в консоль в формате DataFrame с параметрами модели (рис. 4.8).

```
srluipp@vbox:~/project$ julia --project=. scripts/run_simulation.jl
5x7 DataFrame
Row | id arrival service_start departure wait_time service_time system_t
ime | Int64 Float64 Float64 Float64 Float64 Float64 Float64
-----|-----
1 | 1 0.145185 0.145185 4.19698 0.0 4.05179 4.05
179 |
2 | 3 1.54906 4.19698 4.7594 2.64791 0.562426 3.21
034 |
3 | 4 1.62428 4.7594 5.73755 3.13512 0.978144 4.11
327 |
4 | 2 0.594183 0.594183 9.5467 0.0 8.95251 8.95
251 |
5 | 5 1.6911 5.73755 11.3283 4.04645 5.59079 9.63
724 |
srluipp@vbox:~/project$
```

Рисунок 4.8: Вывод файла run_simulation.jl

График времени, проведенного клиентом (заявкой) в системе (рис. 4.9).

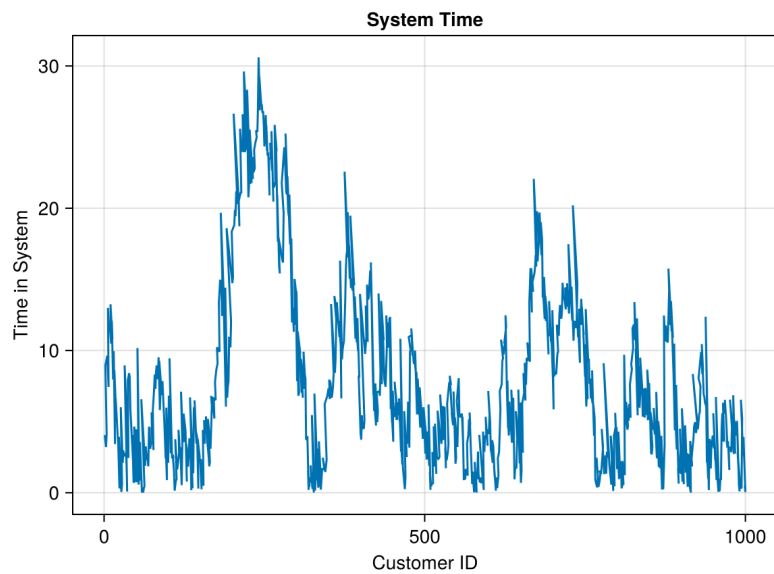


Рисунок 4.9: График времени, проведенного клиентом (заявкой) в системе

График времени ожидания обслуживания (рис. 4.10).

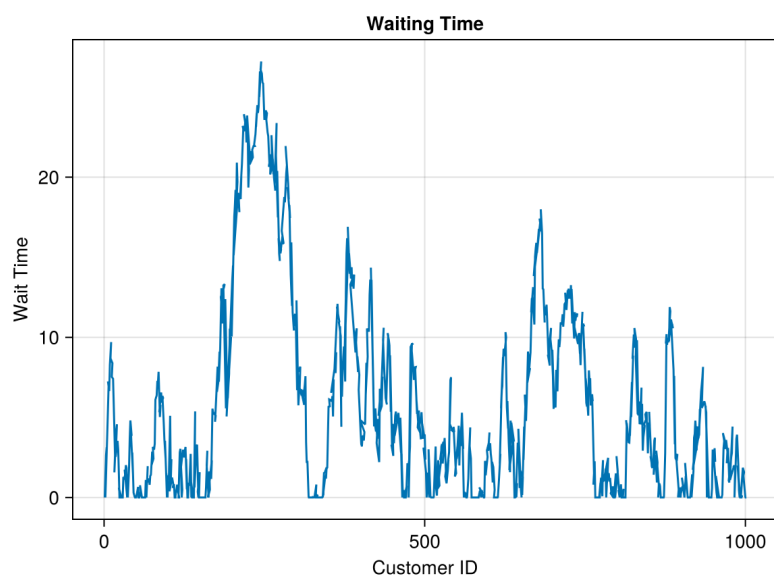


Рисунок 4.10: График времени ожидания обслуживания

График времени ожидания в формате гистограммы (рис. 4.11).

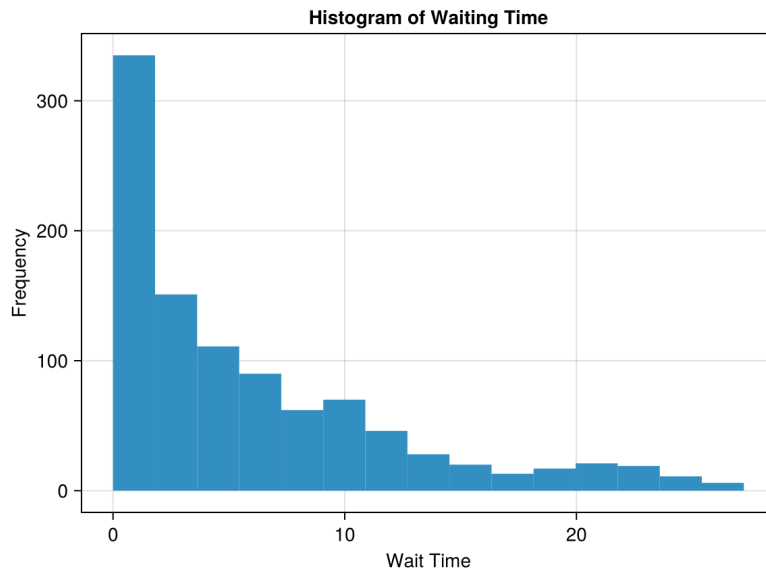


Рисунок 4.11: Гистограмма

Пишу скрипт в `src` (рис. 4.12) для реализации модели Росса и скрипт в `scripts` `run_comparison.jl` с параметрами модели по заданию (рис. 4.13). Добавляю нескольких ремонтников и делаю прогон для разного количества машин (рис. 4.14). Провожу мониторинг загрузки ремонтника, средней длины очереди на ремонт (рис. 4.15). Строю график изменения числа исправных машин во времени и сравниваю с аналитическим решением (рис. 4.16).

```

1 using DrWatson
2
3 @quickactivate "project"
4
5 include(scrdir("simulation2.jl"))
6
7 const RUNS = 10
8
9 results = Float64[]
10
11 for i in 1:RUNS
12     result = sim_repair(
13         N = 10,
14         S = 3,
15         λ = 100.0,
16         μ = 1.0,
17         repairers = 1
18     )
19
20     push!(results, result.crash_time)
21 end
22
23 println("Average crash time = ", mean(results))

```

Рисунок 4.12: Модель Росса скрипт в src

```

srlui@vbox:~/project1$ julia --project=. scripts/run_simulation2.jl
Crash time = 12715.718224958666
Crash time = 12715.718224958666
Crash time = 12715.718224958666
Crash time = 12715.718224958666
Crash time = 12715.718224958666
Crash time = 12715.718224958666
Crash time = 12715.718224958666
Crash time = 12715.718224958666
Crash time = 12715.718224958666
Crash time = 12715.718224958666
Average crash time = 12715.718224958664

```

Рисунок 4.13: Вывод реализации в консоль

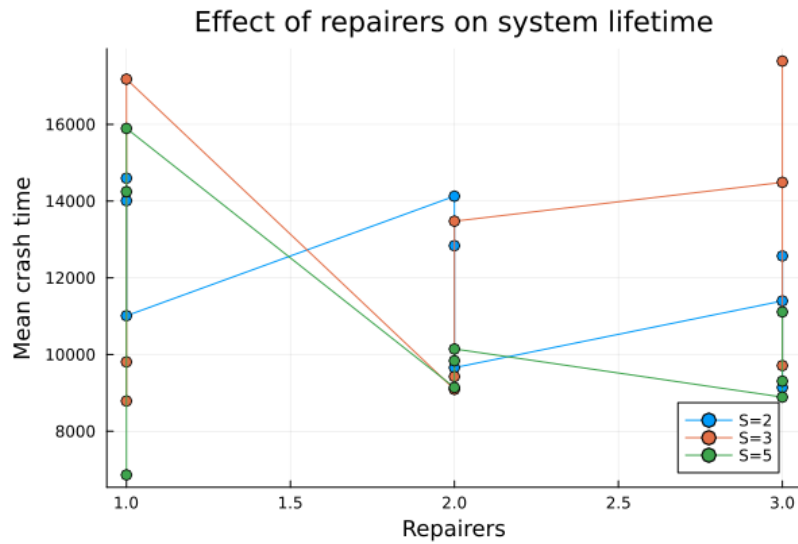


Рисунок 4.14: График для разного количества машин

```
simulationmodeling/labs/lab07/project1$ julia --project=. scripts/run_monitoring.jl
At time 12715.718224958666: No more spares!
At time 37335.53567595007: No more spares!
At time 30844.62667837361: No more spares!
At time 1601.2524911974856: No more spares!
At time 824.1048708405848: No more spares!
Average crash time: 16664.247588264083
At time 17023.673480633508: No more spares!
Crash time: 17023.673480633508
Average queue length: 0.009804892542339308
Repair utilization: 0.06609304922263749
Could not create decoration from factory! Running with no decorations.
Plot saved.
Analytical utilization estimate: 0.05
srluipp@vbox:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--
```

Рисунок 4.15: Вывод реализации в консоль

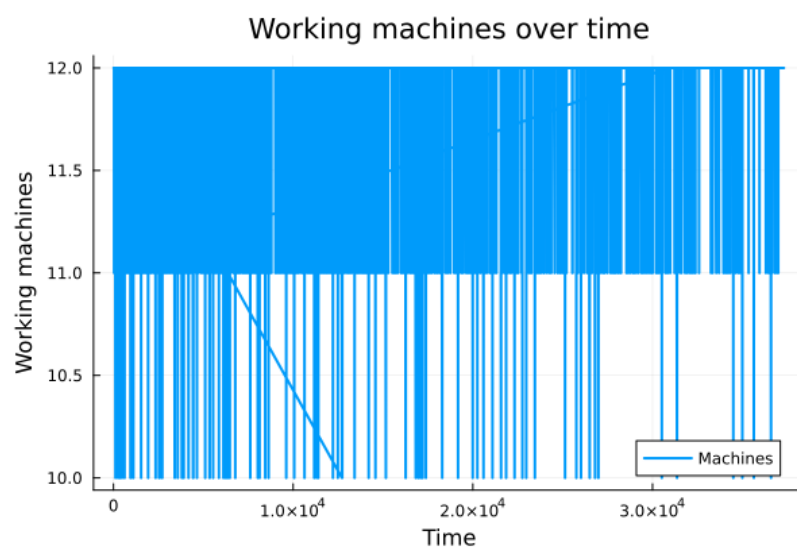


Рисунок 4.16: График изменения числа исправных машин во времени

5 Выводы

В ходе лабораторной работы я изучила дискретно-событийное моделирование и реализовала модель Росса для задачи ремонтников.

Список литературы

- Имитационное моделирование. Практикум А. В. Королькова Кулябов Д. С.